

Product Requirements Document (PRD)

Project Name: AI Store

1. Product Overview

1.1 Objective

The AI Tools Marketplace is a web-based platform designed to help users discover, compare, and effectively utilize modern AI tools. It serves as a curated directory categorizing AI tools (e.g., writing, coding, image generation) and provides deep-dive insights, tutorials, and optimization strategies (prompt engineering and token efficiency).

1.2 Problem Statement

With the rapid proliferation of AI tools, users face decision fatigue when trying to select the right tool for their specific needs. Furthermore, many users lack the technical know-how (such as prompt engineering and token optimization) to use these tools efficiently and cost-effectively.

1.3 Solution

A centralized, SEO-friendly web directory featuring categorized tool listings, dynamic performance leaderboards, and detailed usage tutorials. The platform simplifies discovery and bridges the knowledge gap through step-by-step guides.

2. Target Audience

- **AI Beginners:** Users looking to find the best AI tools for general tasks without feeling overwhelmed.
- **Power Users & Creators:** Individuals looking to optimize their workflow using advanced prompt strategies and token management.
- **Developers & Businesses:** Teams seeking cost-effective APIs and tools for coding, productivity, and research.

3. Core Features & Requirements

3.1 User-Facing Features (Frontend)

- **AI Tools Directory:** * A master feed of all curated AI tools.
 - Card-based UI displaying tool name, logo, short description, and pricing model.
- **Category-Based Navigation:** * Structured taxonomy grouping tools into categories: Writing, Coding, Image Generation, Video Generation, Productivity, and Research.

- **Search and Filtering:** * Keyword-based search bar.
 - Filter tools by category, pricing model (Free, Freemium, Paid), and leaderboard rankings.
- **Dynamic AI Leaderboard:** * A ranking system displaying top tools.
 - Sorting capabilities based on specific evaluation parameters: Capability, Speed, Cost Efficiency, and Usefulness/User Feedback.
- **AI Tool Detail Pages:** * **Overview Section:** In-depth description, website link, limitations, and pricing details.
 - **Education Section:** Embedded tutorials, prompt engineering tips, and token optimization strategies.
- **Tutorials & Usage Guides:** * A dedicated library of step-by-step guides on how to use specific tools effectively.

3.2 Administrator Features (Admin Dashboard)

- **Authentication:** Secure login for platform administrators.
- **Tool Management:** CRUD (Create, Read, Update, Delete) capabilities for AI tool listings.
- **Category Management:** Ability to add or modify categories and map tools to multiple categories.
- **Leaderboard Management:** Interface to manually update or calculate tool ranking scores.
- **Content Management:** An editor (Markdown/MDX support) to write, format, and publish tutorials and usage guides linked to specific tools.

4. System Architecture & Technology Stack

The platform will be built using a modern, serverless-friendly architecture to ensure fast load times, excellent SEO, and scalable database management.

4.1 Tech Stack

- **Frontend:** HTML, CSS, React (JSX), Tailwind CSS, Next.js
- **Backend:** Node.js
- **Database:** PostgreSQL (with SQL for querying)
- **Backend Services / Auth:** Supabase
- **Languages / Formats:** TypeScript (for type safety), Markdown/MDX (for tutorials)
- **Deployment:** Vercel (Frontend) and Supabase/Render (Backend/Database)
- **Version Control:** GitHub

4.2 Data Models (Core Entities)

- **Categories** : Defines the directory taxonomy.
- **Tools** : Stores tool metadata, pricing, and ranking scores.
- **Tool_Categories** : Join table allowing tools to exist in multiple categories.
- **Tutorials** : Stores Markdown content and optimization tips, linked to specific tools.

5. Non-Functional Requirements

- **Performance**: The Next.js frontend must utilize Server-Side Rendering (SSR) and Static Site Generation (SSG) for fast page loads and optimal SEO indexing.
- **Responsiveness**: The UI (using Tailwind CSS) must be fully responsive across mobile, tablet, and desktop devices.
- **Security**: Admin routes must be protected via Supabase authentication. Database row-level security (RLS) should be implemented to prevent unauthorized writes.
- **Scalability**: The database schema should easily accommodate the future addition of user reviews, public authentication, and bookmarking features.

6. Deliverables

1. **Fully Functional Web Platform**: The public-facing Next.js application including the directory, leaderboard, and detail pages.
2. **Admin Management System**: A secure dashboard for content and listing management.
3. **Source Code Repository**: A clean, well-commented GitHub repository with an organized folder structure.
4. **Deployment Setup**: Live hosted environments on Vercel (Frontend) and Supabase (Backend/DB).
5. **Project Documentation**: A **README.md** file containing setup instructions, environment variable configurations, and deployment guidelines.

7. Future Enhancements (Post-MVP)

- Public user authentication to allow users to save/bookmark favorite tools.
- Community-driven reviews and upvoting system.
- API integration to fetch real-time pricing or status updates from popular AI tools.